GomMold

Detect early, breathe easy.

SOFTWARE & AI ENGINEERING PROJECT

윰니 카르미라
누르 샤키라
아이나 눌 인시라
누르 아인 샤피카

INTRODUCTION

PROBLEM

International students and residents in South Korea often struggle to identify early-stage indoor mold due to unfamiliar weather conditions and subtle visual signs.

QUESTION

How can we provide a simple and reliable way for people to detect mold early and receive clear cleaning guidance ?

IDEA

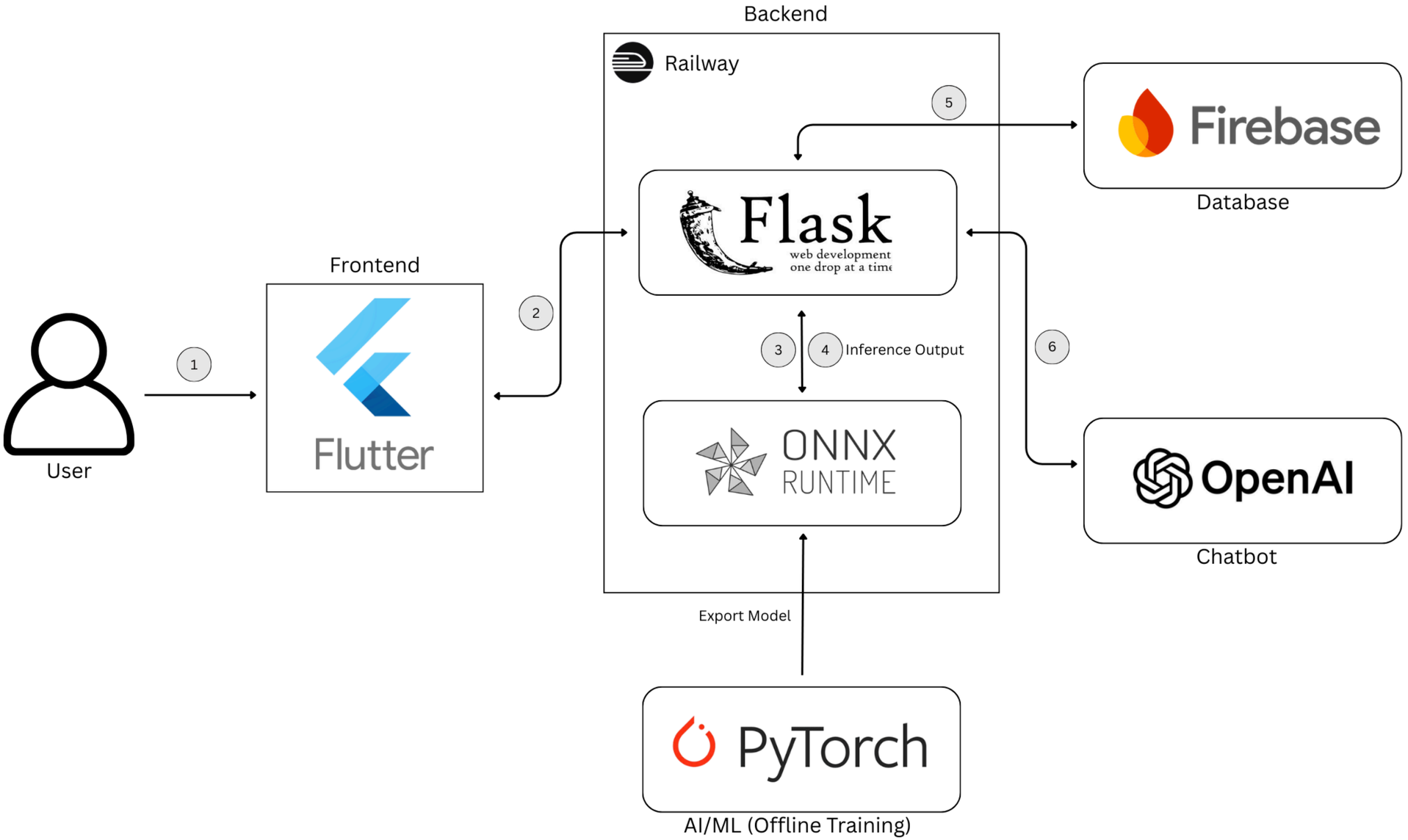
Develop a smartphone app that uses AI to identify mold from photos and instantly offer practical cleaning advice.

GomMold

AI Driven Mold Detection

DEMO VIDEO

ARCHITECTURE



FIRESTORE DATABASE

Users Collection

- email
- username
- password (hashed)
- auto-generated user_id

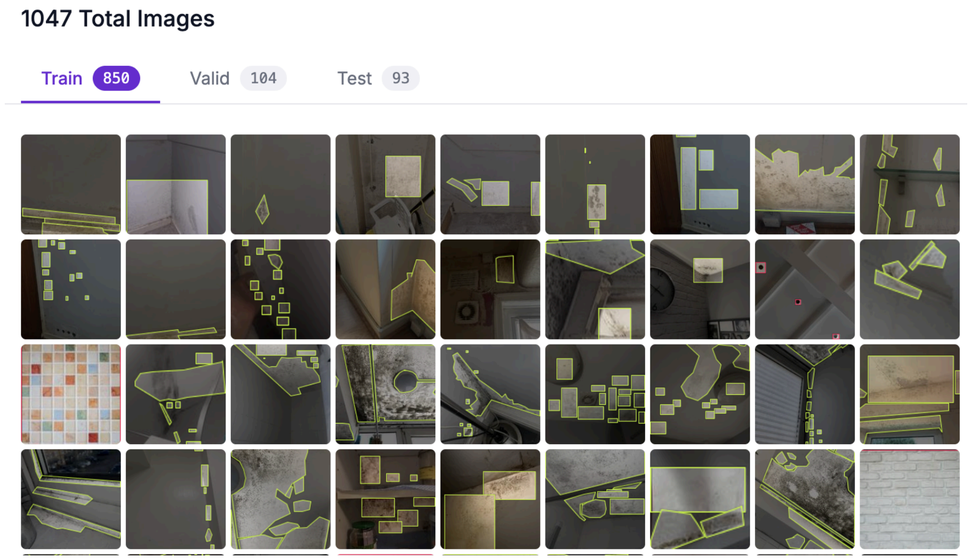
Detections Collection

1. When Mold is Detected
    - result: "warning"
    - message: "Mold detected"
    - predictions: list of bounding boxes (x1, y1, x2, y2, confidence)
    - Color code: Red
    - Saved along with image URL, timestamp, and user ID
2. When No Mold is Detected
    - result: "safe"
    - message: "No mold detected"
    - predictions: empty list []
    - Color code: Green
    - Image still saved, with analysis name and timestamp

DATASET OVERVIEW

Images from the Internet

- 2 classes : mold and no mold
- Images preprocessed + annotated
- Exported into YOLO format



Dataset Structure (YOLO format)

- Dataset Structure (YOLO Format)
  - train/
  - valid/
  - test/
  - data.yaml
- The data.yaml contains:
  - Class names
  - Paths to image directories
  - Number of samples

MODEL CHOICE AND TRAINING

Training

- Loaded yolov8n.pt (COCO pretrained)
- Two classes: mold, no mold
- Tuned epochs, image size, learning rate
- Data augmentation enabled to help the model generalize to different lighting conditions, angles, and wall textures

```
[ ]
from ultralytics import YOLO

model = YOLO("yolov8n.pt")

# Train the model
model.train(
    data="/content/mold_detection_GomMold-4/data.yaml",  # path to dataset
    epochs=50,         # 30–50 recommended
    imgsz=640,         # image size
    batch=16,
    patience=10,       # early stopping if no improvement
    lr0=0.01,          # initial learning rate
    augment=True       # enable augmentation
)

Show hidden output
```

Why YOLOv8n?

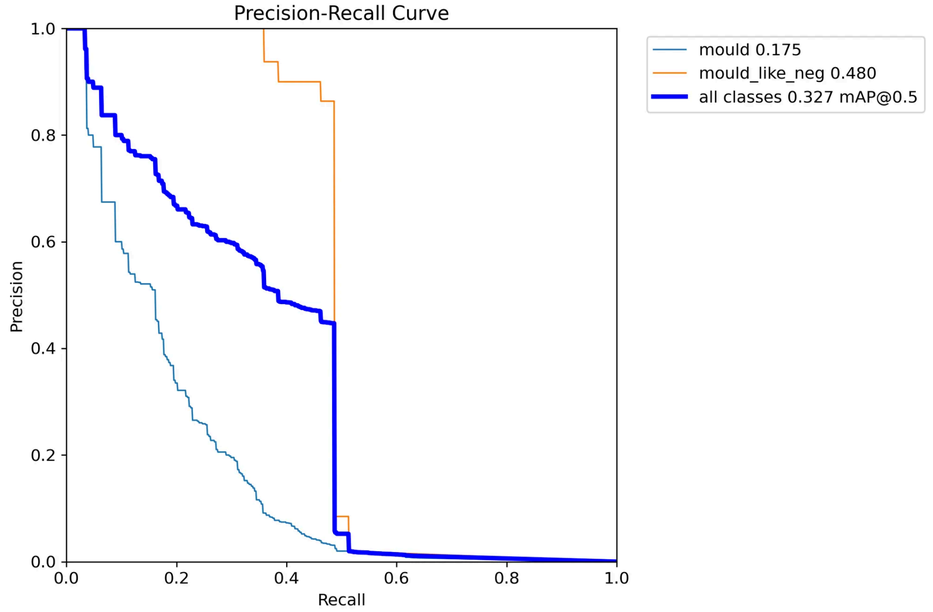
- Lightweight & fast
- Optimized for real-time detection
- Small model size → mobile-friendly
- Works well in cloud environments (Railway)

Transfer Learning

- Start from pretrained COCO model
- Already learns edges, textures, shapes
- Faster training on small dataset
- Higher accuracy vs. training from scratch

MODEL PERFORMANCE

- Works well on clear or larger mold patches.
- Struggles on very small or faint mold spots (sometimes output as nothing or low confidence)



Precision-Recall Curve

- mold AP ≈ 0.175 → lower due to subtle or tiny mold spots
- mold_like_neg AP ≈ 0.480 → performs better because the class is visually consistent
- Overall mAP@50 ≈ 0.327 → acceptable for a small dataset
- Indicates the model can distinguish mold vs no mold but needs more data for tiny mold areas

BEFORE

AFTER

Overall :

- Performs well on clear or larger mold patches
- Struggles with very small, faint, or low-contrast mold spots
- Many subtle mold areas are predicted as background (but very rare false positives or negatives)
- Low confidence for tiny mold regions lowers overall precision (because mold has many **different forms, colors,** and **shapes**, and some spots are extremely **tiny** or **subtle**)
- Encouraging users to take clear, close photos also reduces errors

DEPLOYMENT WITH ONNX

Why ONNX?

- Faster inference :

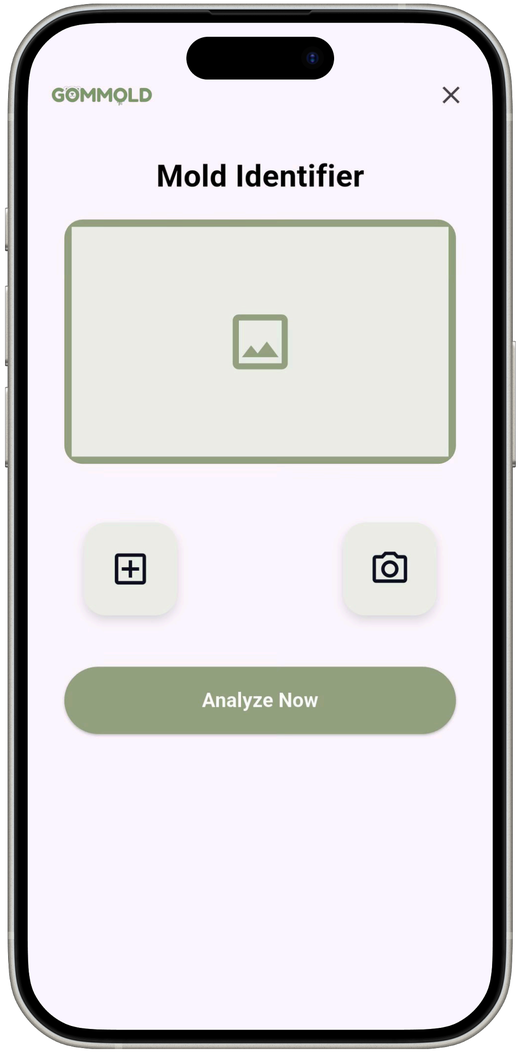
ONNXRuntime is optimized for speed, allowing the model to run predictions much faster than the original PyTorch format.

- Lighter model :

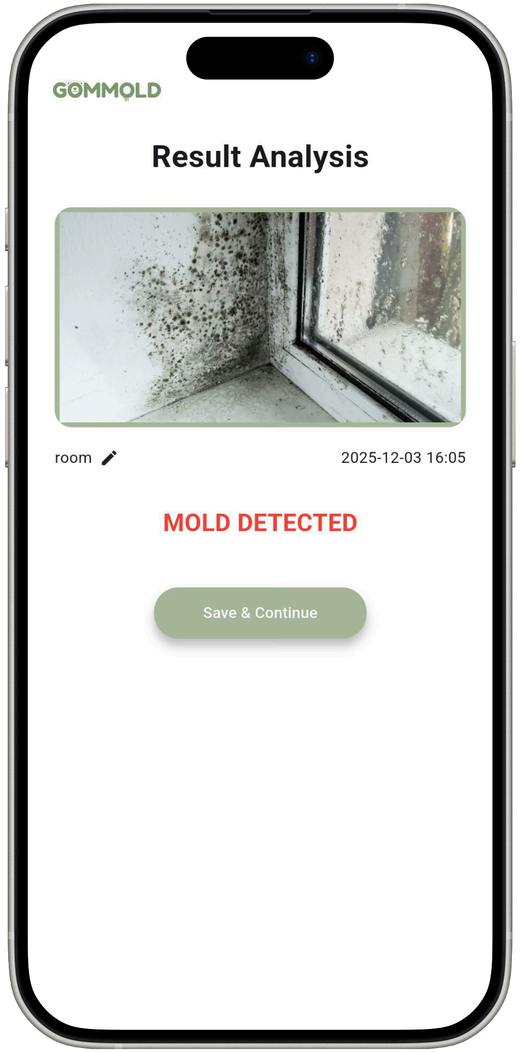
The ONNX format removes training-related components, resulting in a smaller and more efficient file that uses less memory during deployment.

- Works well on Railway backend

Railway

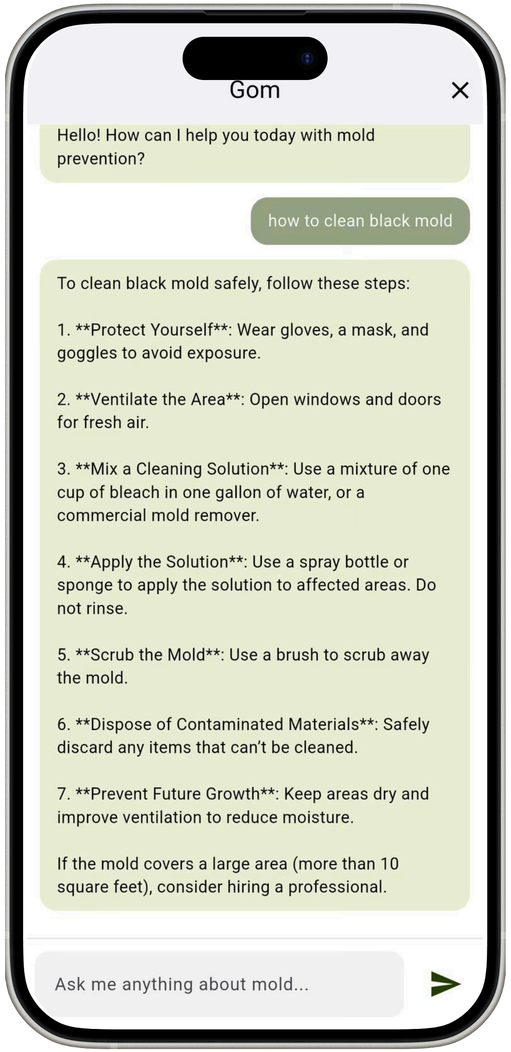


Key feature 1



Key feature 2

Key feature 3



Key feature 4

Thank You